

Content-Based Movie Recommender

Student: Polina Ivanilova | **Team:** Individual

Email: pvivanilova@edu.hse.ru | **Date:** 2026-09-22

Assignment: A02 — Content-Based Movie Recommender (Week 2)

Abstract

I implemented a content-based movie recommender on the MovieLens 100K genre data and addressed five critical issues: garbled titles, an off-by-one genre parse, a broken local fetch, an unusable 1,682-item dropdown, and layout breaks from long titles. To rank movies, I used cosine similarity between the user profile (the mean genre vector of selected films) and each movie's genre vector. This reordered the Star Wars Top-5—Return of the Jedi scored 1.000, while The Empire Strikes Back scored 0.913. Experiments with both single-movie and averaged-profile recommendations returned 0/5 long-tail movies (vs. 22.8% of the catalog), showing that cosine similarity alone does not address the long-tail problem.

Index Terms — content-based recommendation, cosine similarity, user profile, long tail, MovieLens

1. Introduction

Problem statement. The problems split into three classes:

1. *Recommendation problem:*

- *choosing a scoring function.* The first draft compared movies by Jaccard overlap of genre sets. This ignores how many genres a movie has (an 18-genre movie always beats a 1-genre movie) and treats all genres as equally relevant to the user. The task requires a similarity measure and a way to build a *user profile* from more than one watched movie. The adopted solution is cosine similarity between genre vectors, with the profile being the average of the watched movies' vectors.
- *don't re-recommend what is watched.* Already-watched movies are excluded from the results; the watched set is persisted in `localStorage` (`movierec.watched`, and a history on/off flag `movierec.historyOn`) with a reset button.
- *profile vs. single item.* The comparison experiment must show that a single last-watched movie is a weaker profile than the aggregated one, and must be re-runnable for the live user profile with a one-click return to the demo.

2. *Data problem:*

- *the dataset is not UTF-8.* `u.item` is Latin-1. `fetch(url).then(r => r.text())` decodes everything as UTF-8, so accented titles ("Metisse (Café au Lait) (1993)", "Double vie de Véronique, La...") rendered like "M❖❖tisse" until the bytes were decoded with `new TextDecoder('latin1')`.
- *genre columns were misread.* In the first parser the "unknown" genre column (field index 5) was mapped to the label `Action`, shifting every genre one column left. A movie with flags `Action, Adventure` was reported as `unknown, Action`.
- *fetch on file://.* Opening `index.html` directly aborts because Cross-Origin Requests are only supported for protocol schemes like `http/data`; the app has to be served over HTTP (`python -m http.server 8000`).

3. *HTML/UI problem:*

- *long dropdowns.* 1,682 movies in a `<select>` is unusable. Both movie fields became type-ahead `combobox` text inputs with a `None (- no movie -)` option, arrow-key / Enter / Escape handling and "no match" messaging.
- *long titles break tables.* Table cells containing "title (N ratings)" used `white-space: nowrap`; titles like "Shanghai Triad (Yao a yao dao waipo qiao) (1995)" (26 ratings) pushed the table wider than the card. Fix: `white-space: normal`; `word-break: break-word`; `overflow-wrap: anywhere` on all analysis-table cells.

Motivation. This is the second assignment of an AI-assisted Recommender Systems course, which goal is learning basic of item-to-item and user-to-item approaches.

Concrete example. The user types *Toy Story* and *Beauty and the Beast* into the autocomplete fields, clicks *Give a Recommendation*, and gets Top-5/6-10 ranked by cosine against the averaged profile, watched movies excluded.

Contributions.

- A recommender implemented with `index.html`, `style.css`, `data.js`, and `script.js`, without a runtime framework.
- Cosine similarity against an averaged profile of selected movies.
- Type-ahead movie search and optional saved history.
- A comparison between recommendations based on one movie and recommendations based on an aggregated profile.
- Checks of the dataset parsing and a small long-tail analysis.

2. Related Work

Prior work consulted: the MovieLens 100K dataset [1], cosine similarity [2], the Jaccard index [3], the long-tail phenomenon [4], and the Web APIs used for data loading (`TextDecoder` [5], `fetch` [6]). Full references are listed in the References section.

Alternatives considered.

- *Jaccard index* — ignores magnitudes; any same-size genre match ties.
- *Exact-set matching* — near-empty results for multi-genre profiles.

3. Method

Approach. The app loads `u.item` and `u.data` in the browser. I explicitly decode `u.item` as Latin-1 and take the correct genre fields to build a 19-dimensional genre vector for each movie (dimension 0 is “unknown”, followed by the 18 named genres).

For a recommendation, the user can select up to three movies. Their genre vectors are averaged to create a user profile. Then the app calculates cosine similarity between this profile and every movie that is not excluded.

The results are displayed in groups of five. If history is enabled, the selected movies can also be saved in `localStorage`.

Pipeline.

1. `loadData()` fetches and parses `u.item/u.data` (`TextDecoder('latin1')`, `slice(6,24)`), builds rating counts.
2. User picks up to 3 movies via type-ahead search and clicks **Give a Recommendation**.
3. `buildProfileVector` averages the picked genre vectors.
4. `recalculate` scores all non-excluded movies by cosine, sorts descending.
5. The app displays Top-5 and 6-10. When history is enabled, the selected movies are saved.

```
function buildProfileVector(movieList = watchedMovies) {
  const profile = new Array(19).fill(0);
  movieList.forEach(m => m.vector.forEach((v, i) => profile[i] += v / movieList.length));
  return profile;
}

const cosineSimilarity = (a, b) => {
  let dot = 0, na = 0, nb = 0;
  a.forEach((v, i) => { dot += v * b[i]; na += v * v; nb += b[i] * b[i]; });
  return dot / (Math.sqrt(na) * Math.sqrt(nb));
};

function recalculate(profileMovies = watchedMovies) {
  const profile = buildProfileVector(profileMovies);
  const exclude = watchedIds();
  profileMovies.forEach(m => exclude.add(m.id)); // never self-recommend
  rankedMovies = movies
    .filter(m => !exclude.has(m.id))
    .map(m => ({ ...m, score: cosineSimilarity(profile, m.vector) }))
    .sort((a, b) => b.score - a.score);
}
```

Tools & libraries.

- HTML5 / CSS3 / vanilla JavaScript ES6 — no runtime framework.
- MovieLens 100K [1]: 1,682 movies (24 | -separated columns, 19 genre flags), 100,000 ratings.
- Python 3.11.9 for independent verification mirrors; Git / gh for deployment.

Key design decisions.

- *Cosine, not Jaccard*: normalizes by both norms, so a focused match beats a coincidental many-genre one (§4.A).
- *Explicit Latin-1 decode*: `r.text()` replaces byte 0xE9 with U+FFFD; `TextDecoder('latin1')` is lossless.
- *History OFF = one-off*: picks are used for that request only and never recorded or persisted.

Configuration.

- Repo: [PollyIva/Content-Based-Movie-Recommender](#) (public, branch `master`).

4. Experiments

Setup. The behavioral test environment was on the GitHub platform. Three runs: (A) cosine normalization, reference *Star Wars* (1977); (B) demo user **Alex (cartoon fan)**, profile [1,71,95,99,404,50,174,204,588] (Toy Story; Lion King; Aladdin; Snow White; Pinocchio; Star Wars; Raiders; Back to the Future; **Beauty and the Beast** = last = single active item); (C) long-tail thresholds.

Results (A). Star Wars = Action/Adventure/Romance/Sci-Fi/War, $\|v\| = \sqrt{5}$.

Table 1: Genre-share classes vs Star Wars.

Shared genres	Example	raw	cosine
1	Bad Boys (1995)	1	0.447
2	Ghost and the Darkness (1996)	2	0.632
3	Stargate (1994)	3	0.775
4	Starship Troopers (1997)	4	0.894
5 (exact)	Return of the Jedi (1983)	5	1.000
5 (+1 extra)	Empire Strikes Back (1980)	5	0.913

Table 2: Raw vs cosine Top-5.

Rank	Raw order	cosine order
1	Empire Strikes Back (5)	Return of the Jedi (1.000)
2	Return of the Jedi (5)	Empire Strikes Back (0.913)
3	African Queen (4)	African Queen (0.894)
4	Starship Troopers (4)	Starship Troopers (0.894)
5	Abyss (3)	Cutthroat Island (0.775)

Results (B). Both variants exclude all 9 watched movies (Toy Story; Lion King; Aladdin; Snow White; Pinocchio; Star Wars; Raiders; Back to the Future; **Beauty and the Beast** = last = single active item).

Table 3: Experiment Top-5, scores.

Rank	Single item	Aggregated profile
1	All Dogs Go to Heaven 2 (1996) — 1.000	Hercules (1997) — 0.891
2	Cinderella (1950) — 1.000	All Dogs Go to Heaven 2 (1996) — 0.877
3	Alice in Wonderland (1951) — 1.000	Cinderella (1950) — 0.877

Rank	Single item	Aggregated profile
4	Fantasia (1940) — 1.000	Alice in Wonderland (1951) — 0.877
5	James and the Giant Peach (1996) — 1.000	Fantasia (1940) — 0.877

Overlap 4, union 6 → **Jaccard** $\approx$ **0.667**. Unique to single: *James and the Giant Peach*; unique to aggregate: *Hercules*.

Results (C). Rating counts: p25 = 6 (384/1682 $\approx$ 22.8%), p50 = 27 (838 $\approx$ 49.8%).

Table 4: Pick ratings.

Movie	ratings
All Dogs Go to Heaven 2 (1996)	15
Cinderella (1950)	129
Alice in Wonderland (1951)	81
Fantasia (1940)	174
James and the Giant Peach (1996)	126
Hercules (1997)	66

Both methods: **0/5** long-tail at p25 and **1/5** at p50 (All Dogs, 15) — below the 22.8%/49.8% baselines.

Comparison vs. baseline.

(A) Cosine vs. Jaccard:

Return of the Jedi and Star Wars share the same 5 genres, so cosine similarity is 1.000. The Empire Strikes Back has the same 5 genres plus one extra, there are the same number of matches, but due to the additional genre, the vector “lengthens” and the score drops to 0.913. That is, cosine honestly shows that an extra genre does not make the film closer in meaning — it simply expands its genre scope. Therefore, this ranking method provides more meaningful recommendations than just counting how many genres matched.

Table 2 shows the practical effect: cosine similarity changes the order of the results instead of treating all movies with the same overlap as equal.

(B) Single-item vs. aggregated profile:

The Top-5 lists overlap in 4 of 6 unique titles (Jaccard $\approx$ 0.667). James and the Giant Peach appears only for the single-item approach, while Hercules appears only for the aggregated profile. In this case, the recommendations are very similar, because the demo user Alex watches cartoons (similar genres).

(C) Long-tail coverage:

Neither method returns a p25 long-tail movie (0/5), and only one p50 title appears: *All Dogs Go to Heaven 2* (1/5). These rates are below the catalog baselines (22.8% and 49.8%), indicating that cosine similarity alone does not mitigate popularity bias.

Verification. (1) Python mirrors reproduce every score, tie-break, and the Jaccard counts; (2) byte check: 0xE9 renders “Café au Lait” after the fix; (3) counts: 1,682 movies, 100,000 ratings; (4) manual click-through of search, pagination, history on/off/reset, comparison toggle, and long-title wrapping — no console errors; (5) headless-Chrome end-to-end run of the one-off flow confirms history OFF records nothing (`watchedMovies` stays 0, `localStorage['movierec.watched']` stays null) while still returning Top-5.

5. Discussion

Failure cases → detection → fix → confirmation.

- Mojibake titles* — “Metisse (Café au Lait)” showed “M♦tisse”; byte check → `TextDecoder('latin1')` → title rendered.
- Off-by-one genres* — the “unknown” flag column was labeled *Action*, shifting every movie; `slice(6,24)` → spot-checked on 10 known films.
- Blank page on file://* — `fetch` [6] blocked by browser policy; fix: serve over HTTP, documented in README.

4. *Unusable dropdown* — 1,682 `<option>`s; replaced by a type-ahead combobox (with `None`).
5. *Breaking table layout* — “Shanghai Triad ... (1995) (26 ratings)” overflowed under `nowrap`; `word-break / overflow-wrap` → wraps to two lines.
6. *History OFF still recorded* — picks were appended to the in-session profile; now one-off mode keeps the profile empty and saves nothing, confirmed by the headless E2E run (Verification 5).
7. *Stale deployed script (cache)* — after the fix, users still saw picks recorded because the browser served an old cached `script.js`; a static site without cache-busting has no freshness guarantee for its assets. Fix: versioned query strings (`data.js?v=2`, `script.js?v=2`); confirmed by reloading the live page and re-running the E2E scenario.

What worked.

- Latin-1 decode is one lossless line; cosine needs no libraries and its tie-reordering is exactly the teaching point.
- Python mirrors caught a tie-break (id vs title) discrepancy a browser test would miss.
- One-off history mode separates accumulated taste from fresh picks without polluting the saved profile.

What surprised me.

- The data and layout bugs were all *silent* — no console errors, like the a01 Font Awesome case.
- How poorly the recommender surfaced long-tail content — even a random selection would have included more niche titles.

Next improvement. Weight the profile with actual ratings (`u.data`) and evaluate by leave-one-out NDCG; add genre weighting (TF-IDF-like) and a diversity cap to counter the measured bias.

6. AI Usage Disclosure

AI tools used. OpenCode (AI coding CLI, model big-pickle) — categories: code generation, debugging, documentation, visualization assembly.

What I personally verified.

- Manually tested the application in a browser and confirmed that Movie Recommender works correctly (search, recommendations, pagination, history on/off/reset, comparison toggle, long-title wrapping)
- Partially rewrote the report.

References

- [1] F. M. Harper and J. A. Konstan, [“The MovieLens Datasets: History and Context,”](#) *ACM Trans. Interactive Intelligent Systems*, vol. 5, no. 4, 2015. [Online]. Available: <https://grouplens.org/datasets/movielens/100k/> (accessed 2026-09-22).
- [2] Wikipedia, [“Cosine similarity,”](#) Wikimedia Foundation, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Cosine_similarity (accessed 2026-09-22).
- [3] Wikipedia, [“Jaccard index,”](#) Wikimedia Foundation, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Jaccard_index (accessed 2026-09-22).
- [4] C. Anderson, [The Long Tail](#). New York, NY, USA: Hyperion, 2006. [Online]. Available: https://en.wikipedia.org/wiki/The_Long_Tail (accessed 2026-09-22).
- [5] MDN Web Docs, [“TextDecoder — Web APIs,”](#) Mozilla Developer Network, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/TextDecoder> (accessed 2026-09-22).
- [6] MDN Web Docs, [“fetch\(\) — Global fetch\(\) API,”](#) Mozilla Developer Network, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API (accessed 2026-09-22).